Uniswap: Liquidity Launcher

Security Review

Cantina Managed review by:
Devtooligan, Lead Security Researcher
Drastic Watermelon, Security Researcher

June 10, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Uniswap is an open source decentralized exchange that facilitates automated transactions between ERC20 token tokens on various EVM-based chains through the use of liquidity pools and automatic market makers (AMM).

From May 15th to May 21st the Cantina team conducted a review of continuous-clearing-auction on commit hash 68e72ac2. The team identified a total of **10** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	2	2	0
Low Risk	1	1	0
Gas Optimizations	0	0	0
Informational	7	6	1
Total	**10**	**9**	**1**

3 Findings

3.1 Medium Risk

3.1.1 Partial Force Iteration Can Reject Otherwise Valid Final-Block Bids

Severity: Medium Risk

Context: ContinuousClearingAuction.sol#L447

Summary: `forceIterateOverTicks()` can commit an unbracketed clearing price during partial tick iteration. The stored `clearingPrice` can become higher than `nextActiveTickPrice`, causing otherwise valid same-block bids to revert.

Description: `forceIterateOverTicks(_untilTickPrice)` allows public callers to advance tick iteration in bounded chunks. However, the partial iteration path can stop before consuming `_untilTickPrice` and still return the calculated continuous clearing price.

The continuous clearing price calculation is only valid once the iterator has bracketed the price between:

1. The last tick where demand is sufficient to clear the remaining supply, and.
2. The next active tick where demand is no longer sufficient.

Partial force iteration can violate this precondition. If iteration stops because the current `nextActiveTickPrice` equals `_untilTickPrice`, `_iterateOverTicksAndFindClearingPrice()` may return a calculated price above that still-active tick. `forceIterateOverTicks()` then writes that returned value to `$clearingPrice`.

This can leave storage in an inconsistent state:

```
clearingPrice        = P3
nextActiveTickPrice  = P2
```

Here, `P2` is still an active tick, but the stored clearing price has already been raised above it. This breaks the auction's clearing-price monotonicity and tick-bracketing guarantees. The clearing price is expected not to decrease, but this inflated value can later be corrected downward once the still-active lower tick is consumed by a later full iteration or checkpoint.

A malicious caller can trigger the issue through normal public entrypoints:

1. The auction has initialized ticks at `P2` and above.
2. The current block has already been checkpointed.
3. The caller invokes `forceIterateOverTicks(P2)`.
4. The function stops before consuming `P2`, but returns and stores `clearingPrice = P3`.
5. A subsequent same-block bidder submits with `_maxPrice = P3`.
6. `_checkpointAtBlock(block.number)` returns the existing checkpoint, so the overstated price is not normalized.
7. `_submitBid()` checks `_maxPrice <= $clearingPrice` and reverts with `BidMustBeAboveClearingPrice`.
8. A later full iteration consumes `P2` and lowers `clearingPrice` back to `P2`, confirming that the earlier `P3` value was not a valid stable clearing price.

As a result, chunked iteration is not economically equivalent to full iteration. A bid that would be accepted after full iteration can be rejected solely because another caller partially forced iteration first.

Impact Explanation: Impact is Medium. This issue allows bid-admission and auction-outcome manipulation, but does not directly transfer funds from users. Any public caller can temporarily overstate the live bid gate and cause valid bids at or below the inflated `clearingPrice` to revert.

The impact is strongest in the final biddable block. In that case, the rejected bidder cannot retry in a later biddable block after the state normalizes. A valid final-block bid can be suppressed, causing the auction to raise less currency and potentially fail graduation when it otherwise would have graduated.

Likelihood Explanation: Likelihood is Moderate. The issue is reachable through public calls and does not require privileged access. The required state is specific but realistic: the auction must have initialized ticks such that partial iteration stops before a tick that should be consumed before the returned clearing price is valid.

Exploitation also requires timing. The most practical case is a same-block sequence where the block has already been checkpointed, an attacker calls `forceIterateOverTicks()` with a partial bound, and a victim's bid is then evaluated against the overstated stored price. This is most valuable near the end of the auction, especially in the final biddable block where rejected bids cannot be resubmitted later.

Recommendation: Do not allow partial tick iteration to commit a `clearingPrice` unless the persisted tick state brackets that price.

At minimum, after any committed force-iteration update, the invariant `clearingPrice < nextActiveTickPrice` should hold.

Uniswap Labs: Fixed in PR 355.

Cantina Managed: Fix verified.

3.1.2 A bidder can use full force iteration to exclude later same-block bids at the final clearing tick

Severity: Medium Risk

Context: ContinuousClearingAuction.sol#L432

Summary: A bidder can submit at the marginal clearing tick and immediately call `forceIterateOverTicks(MAX_TICK_PTR)`, causing a later same-block bid at the same price to revert. Final clearing price and currency raised remain coherent, but the first bidder captures allocation that would otherwise have been shared pro-rata at the clearing tick.

Description: The auction relies on checkpoints to determine bid fills. A checkpoint is a top-of-block snapshot and does not include the bid that caused it to be written. At most one checkpoint is written per block, so later checkpoint calls in the same block return the existing checkpoint.

When a user submits a bid, `_submitBid` first checkpoints the auction and then validates the new bid against live storage `$clearingPrice`:

```
if (_maxPrice <= $clearingPrice) revert BidMustBeAboveClearingPrice();
```

This creates an admission asymmetry when `forceIterateOverTicks(MAX_TICK_PTR)` is called in the same block as a new bid.

A malicious bidder can:

1. Submit a bid at price `P`.
2. Immediately call `forceIterateOverTicks(MAX_TICK_PTR)`.
3. Cause full force iteration to use the bidder's live same-block demand while still using the already-created top-of-block checkpoint's `cumulativeMps`.
4. Update live storage `$clearingPrice` to `P`.
5. Cause a later same-block bid at the same price `P` to revert with `BidMustBeAboveClearingPrice`.

In the no-force path, the later equal-price bidder would be admitted into the same final clearing tick and would share the tick allocation pro-rata. The issue is therefore not incorrect final price computation or broken currency accounting. The issue is that full force iteration can update the same-block bid admission gate using demand that was excluded from the current block checkpoint.

This is also not merely normal transaction ordering. Alice submitting before Bob does not, by itself, exclude Bob. In the control branch of the PoC, Alice and Bob submit at the same price in the same block and both are accepted. Bob is only excluded when Alice performs full force iteration between the two equal-price submissions.

This issue is distinct from the previously known partial-force issue. The partial-force issue uses a bounded `_untilTickPrice` and can leave the auction temporarily reporting a clearing price above an unprocessed lower tick. This issue uses `_untilTickPrice == MAX_TICK_PTR`, so tick iteration is full and the final clearing price remains coherent. The problem is same-block admission and allocation, not an unprocessed tick frontier.

Impact Explanation: The impact is Medium. A bidder can monopolize scarce marginal clearing-tick allocation against later same-block equal-price bidders. In the PoC, both branches end with the same final clearing price and the same total currency raised, but allocation changes materially:

```
Control path:
Alice receives 500e18 tokens
Bob receives   500e18 tokens

Attack path:
Alice receives 1000e18 tokens
Bob receives      0 tokens because his bid is never admitted
```

The attacker does not receive free tokens and does not corrupt final accounting. However, the attacker can prevent another bidder from participating at the same final clearing tick and capture the allocation that would otherwise have been shared. This breaks the expected pro-rata sharing behavior for equal-price bidders at the clearing price.

The impact is strongest at `END_BLOCK - 1`, because the excluded bidder cannot retry at the same price in a later biddable block.

Likelihood Explanation: The likelihood is Medium. The attacker only needs to bid at the marginal clearing tick and bundle the bid with a full force iteration call.

The main practical constraint is timing. The attack is most useful when a bidder can act in the same block before other equal-price bidders, especially near the end of the auction where retry opportunities are limited. This makes the issue dependent on transaction ordering, but the additional exclusion power comes from full force iteration, not from ordering alone.

Proof of Concept: Add the following test as `test/poc/NewResearcherFindings.t.sol`.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {ContinuousClearingAuction} from '../../src/ContinuousClearingAuction.sol';
import {AuctionParameters, IContinuousClearingAuction} from
↪  '../../src/interfaces/IContinuousClearingAuction.sol';
import {ConstantsLib} from '../../src/libraries/ConstantsLib.sol';
import {FixedPoint96} from '../../src/libraries/FixedPoint96.sol';
import {AuctionStepsBuilder} from '../utils/AuctionStepsBuilder.sol';
import {Test} from 'forge-std/Test.sol';
import {ERC20Mock} from 'openzeppelin-contracts/contracts/mocks/token/ERC20Mock.sol';

contract SameBlockFullForceAttacker {
    function submitAndForce(
        ContinuousClearingAuction auction,
        uint256 price,
        uint128 amount,
        address owner,
```

```
        uint256 prevTick
    ) external payable returns (uint256 bidId) {
        bidId = auction.submitBid{value: amount}(price, amount, owner, prevTick,
        ↪  bytes(''));
        auction.forceIterateOverTicks(auction.MAX_TICK_PTR());
    }
}

contract NewResearcherFindingsTest is Test {
    using AuctionStepsBuilder for bytes;

    uint128 internal constant TOTAL_SUPPLY = 1000e18;
    uint64 internal constant AUCTION_DURATION = 10;
    uint256 internal constant FLOOR_PRICE = 1000 << FixedPoint96.RESOLUTION;
    uint256 internal constant TICK_SPACING = 100 << FixedPoint96.RESOLUTION;
    uint24 internal constant MPS_PER_BLOCK = uint24(ConstantsLib.MPS / AUCTION_DURATION);

    address internal alice = makeAddr('newResearchAlice');
    address internal bob = makeAddr('newResearchBob');
    address internal fundsRecipient = makeAddr('newResearchFundsRecipient');
    address internal tokensRecipient = makeAddr('newResearchTokensRecipient');

    function test_PoC_sameBlockFullForceMonopolizesEqualPriceTick_LastBiddableBlock()
    ↪  public {
        uint256 price = FLOOR_PRICE + TICK_SPACING;
        uint128 amount = _amountForSupplyAt(price);

        (, ContinuousClearingAuction control,) = _deployNativeAuction(amount);
        (, ContinuousClearingAuction attacked,) = _deployNativeAuction(amount);
        SameBlockFullForceAttacker attacker = new SameBlockFullForceAttacker();

        uint64 lastBiddableBlock = control.endBlock() - 1;
        uint256 halfSupply = uint256(TOTAL_SUPPLY) / 2;

        vm.deal(address(this), uint256(amount) * 4);
        vm.roll(lastBiddableBlock);

        uint256 controlAliceBid = control.submitBid{value: amount}(price, amount, alice,
        ↪  FLOOR_PRICE, bytes(''));
        uint256 controlBobBid = control.submitBid{value: amount}(price, amount, bob,
        ↪  FLOOR_PRICE, bytes(''));

        uint256 attackAliceBid = attacker.submitAndForce{value: amount}(attacked, price,
        ↪  amount, alice, FLOOR_PRICE);

        vm.expectRevert(IContinuousClearingAuction.BidMustBeAboveClearingPrice.selector);
        attacked.submitBid{value: amount}(price, amount, bob, FLOOR_PRICE, bytes(''));

        vm.roll(control.endBlock());
        control.checkpoint();
        attacked.checkpoint();

        control.exitPartiallyFilledBid(controlAliceBid, lastBiddableBlock, 0);
        control.exitPartiallyFilledBid(controlBobBid, lastBiddableBlock, 0);
        attacked.exitPartiallyFilledBid(attackAliceBid, lastBiddableBlock, 0);

        assertEq(control.clearingPrice(), price, 'control final price');
        assertEq(attacked.clearingPrice(), price, 'attack final price');
        assertEq(control.currencyRaised(), attacked.currencyRaised(), 'same currency
        ↪  raised');
        assertEq(control.bids(controlAliceBid).tokensFilled, halfSupply, 'control Alice
        ↪  fill');
        assertEq(control.bids(controlBobBid).tokensFilled, halfSupply, 'control Bob
        ↪  fill');
        assertEq(attacked.bids(attackAliceBid).tokensFilled, uint256(TOTAL_SUPPLY),
        ↪  'attack Alice fill');
```

```
    }

    function _deployNativeAuction(uint128 requiredCurrencyRaised)
        internal
        returns (ERC20Mock token, ContinuousClearingAuction auction, uint64 startBlock)
    {
        startBlock = uint64(block.number + 1);
        token = new ERC20Mock();

        AuctionParameters memory params = AuctionParameters({
            currency: address(0),
            tokensRecipient: tokensRecipient,
            fundsRecipient: fundsRecipient,
            startBlock: startBlock,
            endBlock: startBlock + AUCTION_DURATION,
            claimBlock: startBlock + AUCTION_DURATION + 1,
            tickSpacing: TICK_SPACING,
            validationHook: address(0),
            floorPrice: FLOOR_PRICE,
            requiredCurrencyRaised: requiredCurrencyRaised,
            auctionStepsData: AuctionStepsBuilder.init().addStep(MPS_PER_BLOCK,
            ↪  uint40(AUCTION_DURATION))
        });

        auction = new ContinuousClearingAuction(address(token), TOTAL_SUPPLY, params,
        ↪  address(0));
        token.mint(address(auction), TOTAL_SUPPLY);
        auction.onTokensReceived();
    }

    function _amountForSupplyAt(uint256 price) internal pure returns (uint128) {
        return uint128((uint256(TOTAL_SUPPLY) * price) / FixedPoint96.Q96);
    }
}
```

Recommendation: Ensure that demand inserted after the current block checkpoint cannot update the same block's bid admission gate against later equal-price bidders. The fix should preserve the liveness role of `forceIterateOverTicks` while preventing uncheckpointed same-block demand from creating admission asymmetry.

Uniswap Labs: Fixed in PR 355.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 `ContinuousClearingAuction.sweepCurrency` can be DoS'd by a misbehaving protocol fee controller

Severity: Low Risk

Context: ContinuousClearingAuction.sol#L666-L668

Description: When `ContinuousClearingAuction.sweepCurrency` queries the protocol fee controller at sweep time to determine the fee owed on the raised currency, the subtraction `currencyRaised - protocolFeeAmount` performs no bounds check on the value returned by the controller. If a misbehaving or misconfigured `PROTOCOL_FEE_CONTROLLER` returns a `protocolFeeAmount` greater than `currencyRaised`, the subtraction underflows and reverts.

Recommendation: Clamp the fee amount to `currencyRaised` so the sweep can always proceed:

```
  uint256 currencyRaised = currencyRaised();
  uint256 protocolFeeAmount =
      ProtocolFeeLib.getProtocolFeeAmount(PROTOCOL_FEE_CONTROLLER,
      ↪  Currency.unwrap(CURRENCY), currencyRaised);
+ if (protocolFeeAmount > currencyRaised) protocolFeeAmount = currencyRaised;
  _sweepCurrency(_getBlockNumberish(), currencyRaised - protocolFeeAmount);
```

Uniswap Labs: Fixed in PR 354.

Cantina Managed: Fix verified.

3.3 Informational

3.3.1 `ContinuousClearingAuctionFactory` exposes the protocol fee controller via two redundant public getters

Severity: Informational

Context: ContinuousClearingAuctionFactory.sol#L72-L74

Description: `ContinuousClearingAuctionFactory` declares the protocol fee controller as a public immutable and implements the `protoclFeeController()` getter method, giving the contract two distinct view methods which return the same value.

Recommendation: Downgrade the immutable's visibility from `public` to `internal` or `private` and keep the explicit `protocolFeeController()` method as the sole accessor.

Uniswap Labs: Fixed in commit 80937072.

Cantina Managed: Fix verified.

3.3.2 Missing natspec parameter documentation

Severity: Informational

Context: ContinuousClearingAuction.sol#L173-L177

Description: Natspec documentation for the

```
ContinuousClearingAuction._iterateOverTicksAndFindClearingPrice
```

method is missing a `@param` definition for the new `_cumulativeMps` parameter.

Recommendation: Add the missing documentation.

Uniswap Labs: Fixed in PR 353.

Cantina Managed: Fix verified.

3.3.3 `AuctionStateLens.state` can't throw `CheckpointFailed` error

Severity: Informational

Context: AuctionStateLens.sol#L25-L30

Description: When `auction.checkpoint()` reverts inside `revertWithState()`, the contract reverts with `CheckpointFailed`, whose ABI encoding is the 4-byte selector alone.

`state` catches this revert and forwards the reason to `parseRevertReason`, which only bubbles up reasons whose length is greater than 32 bytes. A 4-byte `CheckpointFailed` selector falls through both branches and causes `state` to revert with `InvalidRevertReasonLength` instead.

As a result, `CheckpointFailed` is declared but unreachable for calls to `state`.

Recommendation: Modify the `parseRevertReason` method to always bubble up short reasons as well.

Uniswap Labs: Fixed in PR 353.

Cantina Managed: Fix verified.

3.3.4 `StepStorage` shouldn't be `abstract`

Severity: Informational

Context: StepStorage.sol#L13

Description: `StepStorage` is declared as `abstract` but implements all of its declared methods, making it unnecessary for it to be declared as such.

Recommendation: Remove `abstract` from the contract's declaration.

Uniswap Labs: Fixed in PR 353.

Cantina Managed: Fix verified.

3.3.5 Comment typo

Severity: Informational

Context: IContinuousClearingAuction.sol#L22

Description: The comment at `IContinuousClearingAuction#L22` has a typo of a missing ”be” between ”can claimed”.

Recommendation: Fix the typo to ”can be claimed”.

Uniswap Labs: Fixed in PR 353.

Cantina Managed: Fix verified.

3.3.6 Unused imports across the codebase

Severity: Informational

Context: ContinuousClearingAuction.sol#L11, ContinuousClearingAuction.sol#L25, IAuctionStorage.sol#L4, IAuctionStorage.sol#L6, CheckpointAccountingLib.sol#L6, DemandLib.sol#L4, ValueX7Lib.sol#L4

Technical Description: Several source files import symbols that are never referenced within a file. The following unused imports were identified:

- `ContinuousClearingAuction` imports `IERC20Minimal` and `BlockNumberish`; neither symbol is referenced in the file. `BlockNumberish` is already inherited transitively through `StepStorage`.
- `IAuctionStorage` imports `Currency` and `IERC20Minimal`, neither of which is referenced.
- `CheckpointAccountingLib` imports `FixedPoint96` but never uses it.
- `DemandLib` imports `ConstantsLib` but never uses it.
- `ValueX7Lib` imports `FixedPointMathLib` but never uses it.

Recommendation: Remove the imports listed above.

Uniswap Labs: Fixed in PR 353.

Cantina Managed: Fix verified.

3.3.7 `ContinuousClearingAuctionFactory` methods can declare `amount` as `uint128`

Severity: Informational

Context: ContinuousClearingAuctionFactory.sol#L25-L29

Description: `ContinuousClearingAuctionFactory.initializeDistribution` and `ContinuousClearingAuctionFactory.getAuctionAddress` accept `amount` as `uint256`, then validate `amount <= type(uint128).max` and downcast to `uint128` before use. Declaring the parameter as `uint128` directly would push the bound check to the ABI layer, removing the runtime check, the `uint128(amount)` cast, and the `InvalidTokenAmount` error.

Recommendation: Change `amount` to `uint128` in both methods and remove the `InvalidTokenAmount` check and the downcasts.

Uniswap Labs: Acknowledged.

Cantina Managed: Acknowledged.